Editor Gráfico 2D en OpenGL

— Laboratorio 6 · UNSA 2026-A

21 de septiembre de 2026

Índice

1. Arquitectura del Proyecto **2**

2. Estructura de Datos (`editor.h`) **3**
2.1. Los tipos fundamentales . . . 3
2.2. Un Shape para todo . . . 3
2.3. El estado global del editor . . . 3

3. Ventana y Coordenadas (`main.cpp`) **4**
3.1. Dos sistemas de coordenadas . . . 4
3.2. Registrar callbacks y ceder el control . . . 4

4. Bucle de Eventos GLUT **6**

5. Herramientas de Dibujo (`input.cpp`) **7**
5.1. Ciclo de vida de una figura . . . 7
5.2. Callback de ratón . . . 7

6. Relleno y Selección **8**
6.1. Fase 6 — Relleno de polígonos . . . 8
6.2. Fase 7 — Selección por distancia mínima . . . 8

7. Transformaciones Homogéneas (`editor.h`) **9**
7.1. Por qué matrices 3×3 en 2D . . . 9
7.2. Pipeline de una transformación . . . 9
7.3. Rotación alrededor del centroide . . . 9

8. Pipeline de Renderizado (`render.cpp`) **11**
8.1. Las capas de `display()` . . . 11
8.2. Primitivas OpenGL por tipo de figura . . . 11
8.3. Preview en tiempo real (rubber-band) . . . 11

9. Referencia Rápida **12**
9.1. Tabla completa de controles . . . 12

1. Arquitectura del Proyecto

El editor se divide en cuatro módulos con responsabilidades independientes que se comunican exclusivamente a través del estado global `ed`.

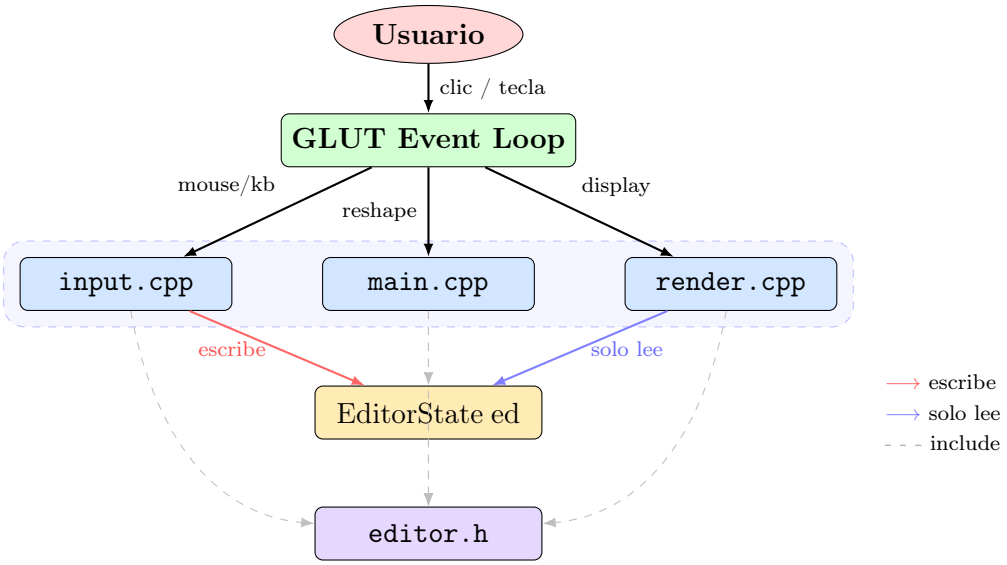


Figura 1: Arquitectura: `input.cpp` es el único módulo que escribe en `ed`; `render.cpp` solo lo lee.

Archivo	Responsabilidad	Escribe en `ed`
`editor.h`	Tipos, estado y matemáticas (no tiene .cpp)	—
`main.cpp`	Inicio, `screenToWorld`, registrar callbacks	crea `ed`
`render.cpp`	Todo el código de dibujo OpenGL	no
`input.cpp`	Callbacks de ratón y teclado	**sí**

2. Estructura de Datos (`editor.h`)

Fase 9 — Cada objeto debe almacenar tipo de figura, lista de vértices, color, etc.

2.1. Los tipos fundamentales

Listing 1: `editor.h` — Vec2, ShapeType y Shape

```
// Coordenada 2D en unidades mundo (no pixeles)
struct Vec2 { float x, y; };

// Un solo tipo de contenedor para TODAS las figuras
enum ShapeType { SHAPE_POINT, SHAPE_LINE, SHAPE_POLYLINE,
                 SHAPE_POLYGON, SHAPE_CIRCLE };

struct Shape {
    ShapeType          type;
    std::vector<Vec2> verts;    // coordenadas que definen la figura
    float r, g, b;              // color del contorno
    float fr, fg, fb;           // color del relleno
    bool  filled   = false;
    bool  selected = false;
};
```

2.2. Un Shape para todo

La misma estructura almacena cualquier figura. Solo cambian `type` y la cantidad de coordenadas en `verts`:



Figura 2: Los cinco tipos comparten el mismo `struct Shape`.

2.3. El estado global del editor

Listing 2: `editor.h` — EditorState: la única fuente de verdad

```
struct EditorState {
    int   W=900, H=650;  float scale=40.f; // ventana y escala
    Mode  modo = MODE_POINT;               // herramienta activa
    std::vector<Shape> escena;     // figuras terminadas
    std::vector<Vec2>  tmpVerts;   // figura en construccion
    Vec2  mouseWorld;              // posicion del cursor
    int   selectedIdx = -1;        // objeto seleccionado
    float colorR, colorG, colorB; // color activo
    float fillR,  fillG,  fillB;  // color relleno activo
};
extern EditorState ed;  // definido en main.cpp, compartido por todos
```

3. Ventana y Coordenadas (`main.cpp`)

Fase 1 — Ventana principal con captura de eventos de teclado y mouse.

3.1. Dos sistemas de coordenadas

GLUT entrega eventos del ratón en **píxeles** (origen arriba-izquierda, Y hacia abajo). OpenGL dibuja en **coordenadas mundo** (origen en el centro, Y hacia arriba). La función `screenToWorld` convierte entre ellos.

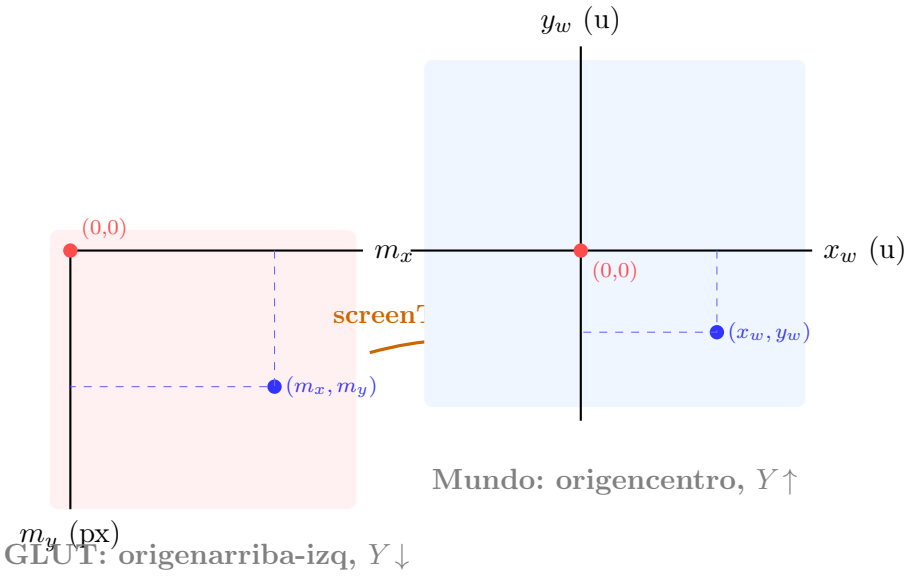


Figura 3: Dos sistemas de coordenadas. `screenToWorld()` hace la conversión.

Listing 3: `main.cpp:17` — Conversión píxel → mundo

```
Vec2 screenToWorld(int mx, int my) {
    return Vec2(
         (mx - ed.W * 0.5f) / ed.scale,   // x mundo
        -(my - ed.H * 0.5f) / ed.scale    // y mundo (Y invertido)
    );
}
// Con W=900, H=650, scale=40:
//   centro (450,325) px  ->  (0, 0) mundo
//   esquina (0,0)    px  ->  (-11.25, +8.125) mundo
```

3.2. Registrar callbacks y ceder el control

Listing 4: `main.cpp:24` — Inicio del editor

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(ed.W, ed.H);
    glutCreateWindow("Editor 2D | ...");

    glutDisplayFunc(display);          // render.cpp: redibujar
    glutReshapeFunc(reshape);          // input.cpp:  ventana resized
    glutMouseFunc(mouse);              // input.cpp:  clic
```

```
    glutKeyboardFunc(keyboard);         // input.cpp:  tecla char
    glutSpecialFunc(specialKey);        // input.cpp:  flechas
    glutPassiveMotionFunc(passiveMotion); // input.cpp: cursor

    glutMainLoop();   // nunca retorna
}
```

4. Bucle de Eventos GLUT

GLUT implementa un **bucle reactivo**: espera eventos del sistema operativo y llama al callback registrado para cada uno. Los callbacks nunca se llaman entre sí — se comunican solo a través de `ed`.

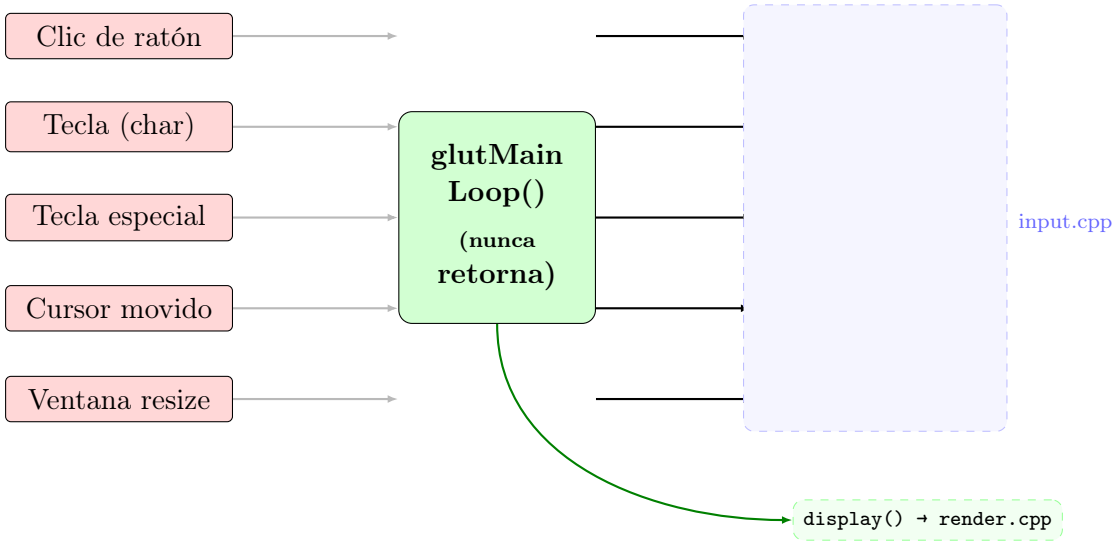


Figura 4: GLUT despacha cada evento al callback correspondiente.

5. Herramientas de Dibujo (`input.cpp`)

Fases 2-5 — Puntos, líneas, polilíneas y polígonos.

5.1. Ciclo de vida de una figura

Cada figura pasa por dos etapas: **construcción** (vértices en `tmpVerts`) y **permanente** (Shape en `escena`).

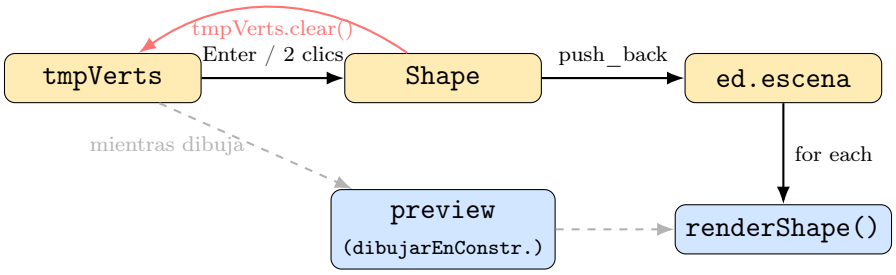


Figura 5: Buffer temporal `tmpVerts` → Shape permanente en `escena`.

5.2. Callback de ratón

Listing 5: `input.cpp:67` — Un switch por herramienta

```
void mouse(int button, int state, int x, int y) {
    if (button != GLUT_LEFT_BUTTON || state != GLUT_DOWN) return;
    Vec2 p = screenToWorld(x, y);          // pixeles -> mundo

    switch (ed.modo) {
        case MODE_POINT:                   // un clic = figura completa
            { Shape s(SHAPE_POINT, ...);
              s.verts.push_back(p);
              ed.escena.push_back(s); }    break;

        case MODE_LINE:                    // necesita 2 clics
            ed.tmpVerts.push_back(p);
            if (ed.tmpVerts.size() == 2) {
                Shape s(SHAPE_LINE, ...);
                s.verts = ed.tmpVerts;
                ed.escena.push_back(s);
                ed.tmpVerts.clear();
            }                              break;

        case MODE_POLYLINE:                // N clics + Enter
        case MODE_POLYGON:
            ed.tmpVerts.push_back(p);      break;

        case MODE_SELECT:
            seleccionar(p);                break;
    }
    glutPostRedisplay();
}
```

6. Relleno y Selección

6.1. Fase 6 — Relleno de polígonos

Activar/desactivar relleno y cambiar colores de contorno e interior.

Los polígonos y círculos tienen dos capas de dibujo: primero el área interior (`GL_POLYGON` o `GL_TRIANGLE_FAN`), luego el contorno encima (`GL_LINE_LOOP`).

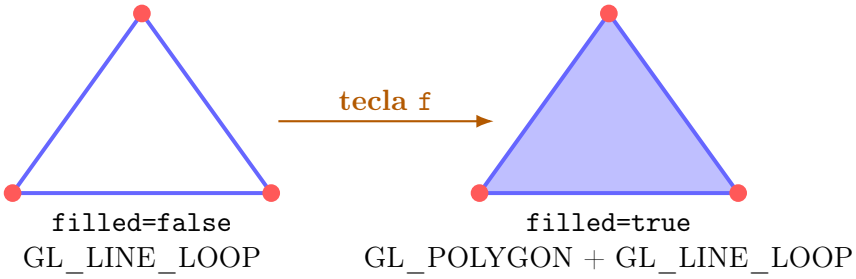


Figura 6: Tecla `f` alterna el relleno; `c`/`v` ciclan los colores.

6.2. Fase 7 — Selección por distancia mínima

Seleccionar objetos con mouse; resaltar visualmente el seleccionado.

Al hacer clic en modo SELECT, se calcula la distancia del clic a cada figura y se elige la más cercana dentro del umbral de 0.6 unidades.

Para **segmentos**, la distancia se calcula proyectando el punto P sobre la recta y clampando $t \in [0, 1]$:

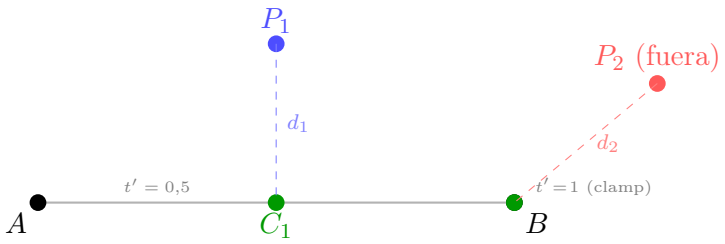


Figura 7: P_1 proyecta dentro del segmento; P_2 clampa al extremo B.

$$t = \frac{(P - A) \cdot (B - A)}{\|B - A\|^2}, \quad t' = \text{clamp}(t, 0, 1), \quad C = A + t'(B - A), \quad d = \|P - C\|$$

El objeto seleccionado se dibuja en **naranja**: `glColor3f(1.0, 0.65, 0.0)`.

7. Transformaciones Homogéneas (`editor.h`)

Fase 8 — Traslación, rotación y escalamiento sobre el objeto seleccionado mediante matrices homogéneas 3×3.

7.1. Por qué matrices 3×3 en 2D

La traslación no cabe en una matriz 2×2. Añadiendo $w = 1$ al punto se puede expresar *toda* transformación afín como una sola multiplicación:

$$\underbrace{\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}}_{\text{Traslación}} \quad \underbrace{\begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}}_{\text{Rotación}} \quad \underbrace{\begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}}_{\text{Escala}} \tag{1}$$

7.2. Pipeline de una transformación

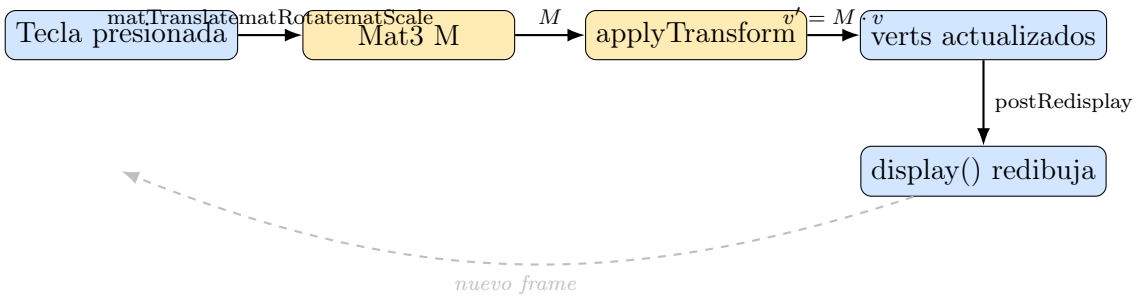


Figura 8: De la tecla al píxel: la transformación modifica `s.verts` directamente.

7.3. Rotación alrededor del centroide

Rotar en torno al centro propio requiere tres pasos compuestos en una sola matriz:

$$M_{\text{rot}}(C, \theta) = \underbrace{T(+C)}_{\text{3. devolver}} \cdot \underbrace{R(\theta)}_{\text{2. rotar}} \cdot \underbrace{T(-C)}_{\text{1. al origen}} \tag{2}$$

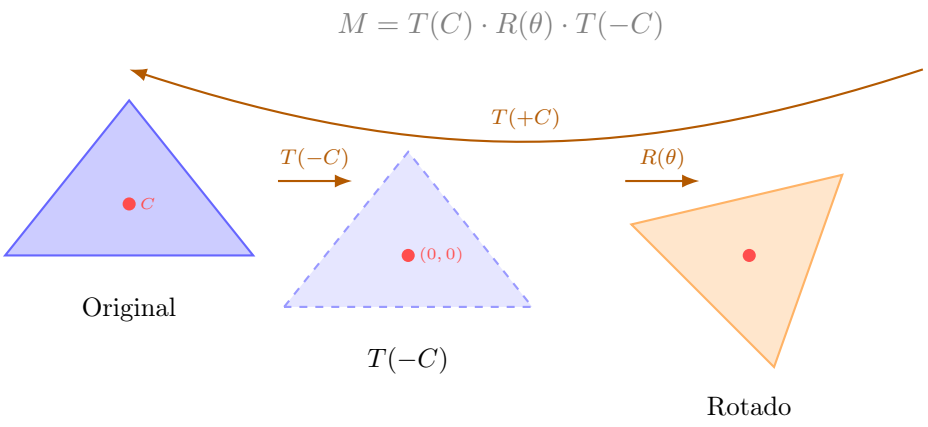


Figura 9: Rotación alrededor del centroide C: trasladar al origen → rotar → devolver.

Listing 6: `editor.h:142` — Implementación de rotación sobre pivote

```
// T(P) * R(a) * T(-P)  en una sola linea
inline Mat3 matRotateAround(Vec2 p, float a) {
    return matTranslate(p.x, p.y) * matRotate(a) * matTranslate(-p.x,-p.y)
        ;
}
// Uso en input.cpp: rotar 5 grados alrededor del centroide
Vec2 c = centroide(s.verts);
applyTransform(s, matRotateAround(c, 5.f * 3.14159f / 180.f));
```

8. Pipeline de Renderizado (`render.cpp`)

8.1. Las capas de `display()`

Cada frame redibuja cuatro capas en orden fijo, de abajo a arriba:

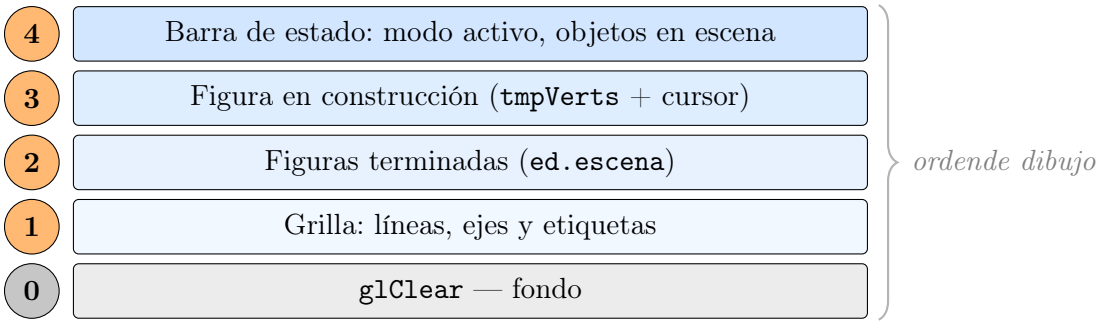


Figura 10: Orden de dibujo en cada llamada a `display()`.

8.2. Primitivas OpenGL por tipo de figura

ShapeType	Primitiva OpenGL	Nota
SHAPE_POINT	GL_POINTS	tamaño 7 px (9 si seleccionado)
SHAPE_LINE	GL_LINES	segmento entre verts[0] y verts[1]
SHAPE_POLYLINE	GL_LINE_STRIP	segmentos conectados, abierto
SHAPE_POLYGON	GL_LINE_LOOP	igual + segmento de cierre
SHAPE_POLYGON	GL_POLYGON	área rellena (solo si `filled`)
SHAPE_CIRCLE	GL_LINE_LOOP	64 segmentos círculo perfecto
SHAPE_CIRCLE	GL_TRIANGLE_FAN	relleno (solo si `filled`)

8.3. Preview en tiempo real (rubber-band)

Cada movimiento del cursor actualiza `ed.mouseWorld` vía `passiveMotion()`, y `dibujarEnConstruccion(` dibuja un segmento provisional hasta esa posición:

Listing 7: `render.cpp:233` — Preview de línea hasta el cursor

```
case MODE_LINE:
    if (ed.tmpVerts.size() == 1) {
        glBegin(GL_LINES);
            glVertex2f(ed.tmpVerts[0].x, ed.tmpVerts[0].y); // punto fijo
            glVertex2f(ed.mouseWorld.x,  ed.mouseWorld.y);  // cursor
                actual
        glEnd();
    }
```

9. Referencia Rápida

9.1. Tabla completa de controles

Acción	Tecla	Qué ocurre en el código
Cambiar herramienta	0–5	`ed.modo` = SELECT/POINT/.../CIRCLE; `tmpVerts.clear()`
Añadir coord	Clic izquierdo	`tmpVerts.push_back(p)` ó `escena.push_back(Shape)`
Finalizar figura	ENTER	Shape sellada → `escena`; `tmpVerts.clear()`
Cancelar	ESC	`tmpVerts.clear()`
Seleccionar	0 + clic	`seleccionar(p)` → `selectedIdx`
Trasladar	←↑→↓	`applyTransform(s, matTranslate(±0.5, ...))`
Rotar	[/]	`applyTransform(s, matRotateAround(c, ±5°))`
Escalar	q / z	`applyTransform(s, matScaleAround(c, 1.1/0.9))`
Toggle relleno	f	`s.filled = !s.filled`
Color contorno	c	cicla paleta → `s.r/g/b`
Color relleno	v	cicla paleta → `s.fr/fg/fb`
Eliminar	Del	`escena.erase(selectedIdx)`